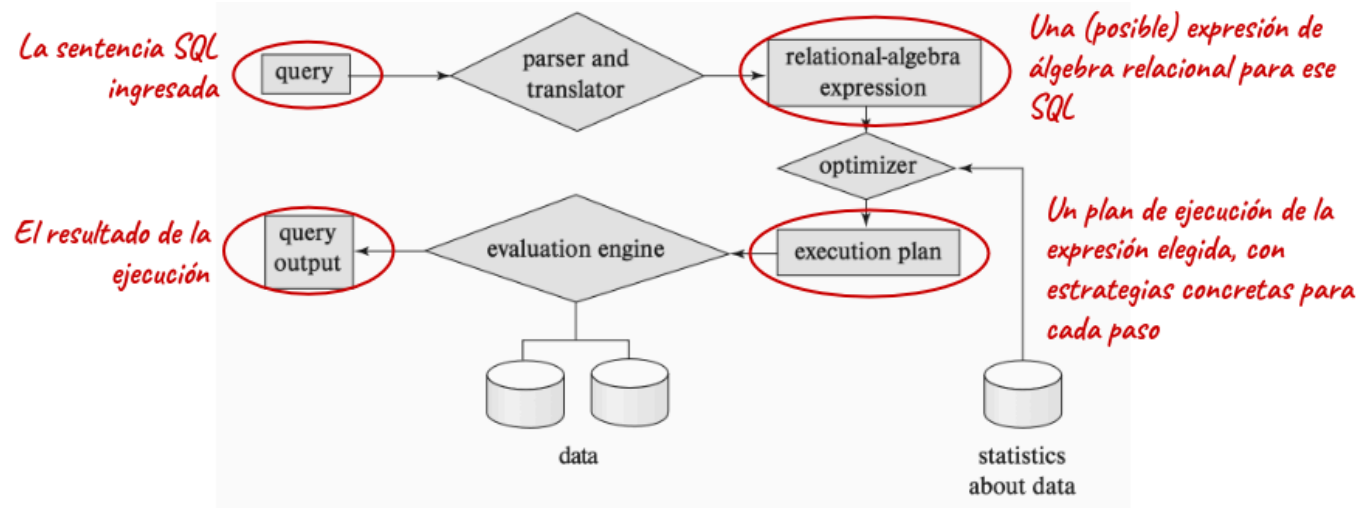


T7.4 - Procesamiento de Consultas

Procesamiento de consultas



1. La entrada es la sentencia SQL. Recordar que *definimos qué queremos*, no *cómo buscar la solución*.
2. Se *traduce la sentencia a álgebra relacional*. Este lenguaje indica *qué pasos se deben seguir* para encontrar la solución.
3. El *motor evalúa distintas expresiones equivalentes y estrategias*. Se queda con la más probable de ser la *óptima*.

Plan de ejecución

Información de catálogo

Las estrategias para ejecutar una consulta van a tener distinto costo.

El motor mantiene **datos estadísticos sobre las relaciones**:

- n_r : *cantidad de registros* (tuplas) en la relación r .
- b_r : *cantidad de bloques con registros* de la relación r .
- l_r : *tamaño de un registro* de la relación r .
- f_r : *cantidad de registros de r que entran en un bloque*.
- $V(A, r)$: *cantidad de valores distintos* que aparecen en el *atributo A de la relación r* .

También hay **datos estadísticos sobre los índices de esas relaciones**:

- $Height_{I(A,r)}$: altura del árbol B+ del índice I sobre el o los atributos A de r .
- $Length_{I(A,r)}$: tamaño en bloques de las hojas del índice I .

Plan de ejecución

Cuando se traduce la sentencia SQL a álgebra, *el motor genera expresiones equivalentes*.

Para cada estrategia se *analizan sus costos usando las estadísticas*, sumado a una **estimación de costo**.

Las estimaciones se basan en las dos operaciones más costosas: **seek de bloque** t_S y **transfer de bloque a memoria** t_T .

El motor, viendo las *estadísticas más las estimaciones y con ayuda de una heurística*, elige un *plan de ejecución* que hace el procedimiento completo incluyendo *qué índices y algoritmos usar*.

Estimación de costo

Estrategias para la selección

Full Scan (búsqueda lineal)

Implica un seek inicial para encontrar el primer bloque de la relación, sumado a la transferencia de todos los bloques de la relación.

$$t_S + b_r \times t_T$$

Si se fuera a dar el caso en el cual la selección se realiza por igualdad con un atributo clave, en promedio, será:

$$t_S + (b_r/2) \times t_T$$

Búsqueda con clustering index

Sobre un atributo clave

Se recorre la altura del árbol hasta encontrar el registro al bloque de la relación.

$$(h_i + 1) \times (t_S + t_T)$$

Leer h_i nodos (uno por nivel) del árbol + 1 bloque de la relación.

Cada nodo del árbol está en un bloque distinto del disco, por lo que hay un seek + un transfer para cada uno.

Sobre un atributo no clave

Notar que *puede haber más de un registro que matchee con el valor de un atributo no clave*.

Se recorre la altura del árbol, hacer un seek para encontrar el primer bloque que coincida y luego transferir todos los otros b bloques con registros para el valor.

$$\underbrace{h_i \times (t_S + t_T)}_{\text{bajar en el árbol}} + \underbrace{t_S + b \times t_T}_{\text{leer todos los bloques de la relación}}$$

Bajar en el árbol → recorrer los niveles del B+ desde la raíz hasta la hoja para *encontrar dónde están los datos en la relación*. Cada nivel está en un bloque distinto del disco, así que para pasar de un nivel al siguiente se hace un seek + transfer.

Leer todos los bloques de la relación → antes, con atributo clave, había un solo registro (un solo bloque a traer a memoria con seek + transfer). Ahora, *puede haber varios que coincidan*.

Como es un clustering index, *esos registros están todos adyacentes en el disco*.

Basta con hacer *seek del primer bloque* y hacer *transfer de la seguidilla* de todos los b bloques.

Búsqueda con secondary index

Sobre un atributo clave

Al igual que con clustering index.

$$(h_i + 1) \times (t_S + t_T)$$

Si el atributo es clave, *hay un solo registro que matchea*.

Esta fórmula incluye:

$$\underbrace{h_i \times (t_S + t_T)}_{\text{bajar en el árbol}} + \underbrace{1 \times (t_S + t_T)}_{\text{buscar registro al bloque}}$$

Como es un solo registro, *no importa si el índice es clustering o secondary*.

La diferencia entre clustering y secondary es cómo están organizados físicamente los registros cuando hay varias que matchean.

Pero si hay una sola, es un solo bloque y listo, no existe una organización contigua o dispersa.

Sobre un atributo no clave

Se recorre la altura del árbol, se hace un seek + transfer para todo registro (n en total) que coincidan.

$$(h_i + n) \times (t_S + t_T)$$

Los registros con `dept_id = 5` pueden estar en cualquier parte del disco. Como la relación está ordenada físicamente según el clustering index (usando otro atributo, supongamos la `PK`), todos los registros que tienen `dept_id = 5` quedaron dispersos, uno puede estar en el bloque 900 y otro en el bloque 50. Si tenemos n registros, la fórmula se descompone en:

$$\underbrace{n \times (t_S + t_T)}_{\text{encontrar y cargar a memoria los registros}} + \underbrace{h_i \times (t_S + t_T)}_{\text{bajar en el árbol}}$$

Búsqueda por comparación (rango)

Con clustering index

Se recorre la altura del árbol, hacer seek para encontrar el primer bloque que hace match y transferir todos los otros b bloques con registros de ese lado del rango.

$$h_i \times (t_S + t_T) + t_S + b \times t_T$$

Es igual al clustering index por atributo no clave.

Para encontrar `suelto > 5000` con clustering index sobre `suelto`:

1. Bajar el árbol hasta encontrar el primer valor mayor a `5000` $\rightarrow h_i \times (t_S + t_T)$.
2. Posicionarse en ese bloque $\rightarrow +t_S$.
3. Leer todos los bloques contiguos desde ahí hasta el final de la relación $\rightarrow b \times t_T$.

Si la búsqueda es por $<$ o $\leq$, no es necesario recorrer el árbol.

$$t_S + b \times t_T$$

Porque como es un clustering index, los registros están ordenados físicamente en disco. Los *valores más chicos están al principio de la relación*. Entonces si se busca `suelto < 3000`:

- 4) Se sabe que los valores `suelto < 3000` están *desde el primer bloque de la relación*.
- 5) No se necesita bajar el árbol para encontrar dónde empieza el rango, porque es desde el principio.
- 6) Se posiciona en el primer bloque (t_S), y se lee de corrido hasta que los valores dejen de cumplir que `suelto < 3000` ($b \times t_T$).

Es como un diccionario, si se buscan las palabras que empiezan con "M" en adelante, busco la "M" primero con el índice y de ahí leo para adelante.

Si se buscan las palabras anteriores a "M", se comienza desde la primera página y se lee hasta llegar a la "M". No se usa el índice.

Con secondary index

Hay que recorrer la altura del árbol, hacer un seek y transfer para todos los registros n que hagan match.

$$(h_i + n) \times (t_S + t_T)$$

Es igual al secondary index sobre un atributo no clave.

Los registros que caen dentro del rango *están dispersos en disco* (justamente por ser secondary index).

Entonces, *por cada registro que matchea implica un seek + transfer.*

No hay un caso especial para $<$ o $\leq$, porque la optimización de "arrancar desde el principio" *solo funciona cuando las tuplas están ordenadas físicamente por ese atributo.*

En un secondary index no lo están, así que *siempre se debe bajar el árbol* para encontrar los punteros a los registros, sin importar la dirección del rango.

Selecciones complejas

Conjunción de condiciones (and)

1. Usar el *índice para alguna de las condiciones*, mejor si es la *más barata*. Se usa alguno de los algoritmos anteriores y se evalúan las demás condiciones con los registros ya en memoria.
2. Usar un *índice compuesto* que *aplique a todas las condiciones*, aplicando alguna de las estrategias anteriores.
3. Usar *índices para muchas condiciones* e intersecar los identificadores.

Disyunción de condiciones (or)

1. Usar índices para todas las condiciones, luego unir los identificadores.
2. Hacer un full scan y evaluar las condiciones en cada tupla en memoria.

Junta

La operación de junta ($\bowtie$) *une tuplas de dos relaciones según una condición*. El motor tiene *varias estrategias* para implementarla, y *cada una tiene un costo distinto dependiendo de la situación* concreta (tamaño de las relaciones, índices disponibles, memoria...)

Las *dos relaciones* involucradas se llaman **exterior** (e) e **interior** (i). En general, conviene que la exterior sea la más chica.

Loop anidado por bloques

Recorre la relación exterior *bloque a bloque*. Por *cada bloque de la exterior*, *recorre toda la relación interior* bloque a bloque. Por cada par de bloques, compara todas las tuplas entre sí.

Para cada bloque B_e de la exterior:

Traer B_e a memoria.

Para cada bloque B_i de la interior:

Traer B_i a memoria.

Comparar todas las tuplas de B_e con las de B_i .

$$\underbrace{b_e}_{\text{bloques de la exterior (se leen una vez)}} \times \left(\underbrace{1}_{\text{leer el bloque de } e} + \underbrace{b_i}_{\text{recorrer toda la interior}} \right) \times (t_T + t_S)$$

- Cada bloque de la exterior se lee una sola vez $\rightarrow b_e$ lecturas.
- Por cada bloque de la exterior, se lee toda la interior $\rightarrow b_e \times b_i$ lecturas.
- Total de bloques leídos $\rightarrow b_e \times (1 + b_i)$.
- *Se asume peor caso*: cada acceso a un bloque es disperso $\rightarrow$ seek + transfer por cada uno.

Supongamos $b_e = 10$, $b_i = 100$:

$$10 \times (1 + 100) \times (t_T + t_S) = 1010 \times (t_T + t_S)$$

Si invirtiéramos las relaciones ($b_e = 100$, $b_i = 10$):

$$100 \times (1 + 10) \times (t_T + t_S) = 1100 \times (t_T + t_S)$$

Conviene poner como exterior la relación más chica ($b_e < b_i$), porque la interior se recorre b_e veces.

Loop indexado

Recorre la relación exterior tupla por tupla. Por cada tupla de la exterior, usa el **índice** de la interior para buscar directamente las tuplas que matchean, en vez de recorrer toda la interior.

Para cada bloque B_e de la exterior:

Para cada tupla te en B_e :

Buscar en el índice de la interior las tuplas que matchean con te .

$$\underbrace{b_e \times (t_T + t_S)}_{\text{leer la exterior bloque por bloque}} + \underbrace{n_e}_{\text{tuplas de la exterior}} \times \underbrace{c}_{\text{costo de buscar una tupla en la interior usando el índice}}$$

- Se leen los b_e bloques de la exterior (*peor caso*: dispersos → seek + transfer cada uno).
- Por cada una de las n_e tuplas de la *exterior*, se hace *una búsqueda por índice en la interior*. El *costo de cada búsqueda* (c) es el mismo que el de selección por índice.
- La *relación interior no se recorre*, se *accede puntualmente por índice* → mucho más barato que BNLJ si el índice es selectivo.

$b_e = 10$, $n_e = 500$, índice sobre clave de la interior con $h_i = 3$:

$$c = (3 + 1) \times (t_T + t_S) = 4 \times (t_T + t_S)$$

$$\text{Total} = 10 \times (t_T + t_S) + 500 \times 4 \times (t_T + t_S) = 2010 \times (t_T + t_S)$$

Sort-merge

1. *Ordena* ambas relaciones por el atributo de la condición de junta (si no lo están ya).
2. *Mergea*: recorre ambas relaciones ordenadas en paralelo, emparejando los registros que matchean.

$$\left(\underbrace{b_e + b_i}_{\text{leer ambas relaciones en el merge final}} + \underbrace{2 \times b_e \times \lceil \log_{M-1}(b_e) \rceil}_{\text{sort externo de la exterior: leer + escribir por cada pasada}} + \underbrace{2 \times b_i \times \lceil \log_{M-1}(b_i) \rceil}_{\text{sort externo de la interior: leer + escribir por cada pasada}} \right) \times (t_T + t_S)$$

- El *sort externo* de una relación de b bloques con M bloques de memoria requiere $\lceil \log_{M-1}(b) \rceil$ pasadas. Cada pasada lee y escribe todos los bloques → $2 \times b$ accesos por pasada.
- Una vez ordenadas, el *merge* recorre ambas relaciones una vez → $b_e + b_i$ accesos.
- Se *asume peor caso*: cada acceso disperso → $(t_T + t_S)$ por bloque.

$b_e = 100$, $b_i = 200$, $M = 11$ bloques de memoria:

$$\text{Sort de } e: 2 \times 100 \times \lceil \log_{10}(100) \rceil = 2 \times 100 \times 2 = 400$$

$$\text{Sort de } i: 2 \times 200 \times \lceil \log_{10}(200) \rceil = 2 \times 200 \times 3 = 1200$$

$$\text{Merge: } 100 + 200 = 300$$

$$\text{Total: } (400 + 1200 + 300) \times (t_T + t_S) = 1900 \times (t_T + t_S)$$

Hash

1. *Particionado*: aplica una función de hash sobre el atributo de junta a ambas relaciones. Los registros se dividen en particiones según el valor del hash. Se

escriben las particiones a disco.

2. **Combinación**: por cada par de particiones con el mismo valor de hash, se hace la junta por fuerza bruta en memoria.

$$\underbrace{2 \times (b_e + b_i) \times (t_T + t_S)}_{\text{particionado: leer + escribir}} + \underbrace{(b_e + b_i) \times (t_T + t_S)}_{\text{combinación: leer particiones}}$$

Particionado:

- Leer todos los bloques de ambas relaciones: $(b_e + b_i)$ accesos.
- Escribir todas las particiones a disco: $(b_e + b_i)$ accesos más.
- Total: $2 \times (b_e + b_i)$.

Combinación:

- Se leen las particiones de ambas relaciones para hacer la junta: $(b_e + b_i)$ accesos.
- La junta de cada par de particiones se hace **en memoria** (por fuerza bruta), así que no hay costo adicional de I/O.

$b_e = 100, b_i = 200$:

$$3 \times (100 + 200) \times (t_T + t_S) = 900 \times (t_T + t_S)$$

Optimización

Cómo seleccionar el plan de ejecución

El optimizador **no busca el mejor plan** posible, sino un **plan eficiente**. Analizar todo el espacio de planes posibles puede ser más costoso que ejecutar la consulta misma.

Buscar el mejor plan puede tardar 10 min y el plan ejecutarse en 1 ms (total ≈ 10 min). Buscar un plan eficiente puede tardar 1 seg y ejecutarse en 1 seg (total = 2 seg).

Para encontrar un plan eficiente, el optimizador **se apoya** en (i) reglas de equivalencia, (ii) heurísticas, (iii) análisis de costo y (iv) estimación de cardinalidades

Reglas de equivalencia

Permiten **transformar** una expresión de álgebra relacional en **otra equivalente** $\rightarrow$ mismo resultado, distinto procedimiento.

Selección

- Condiciones conjuntivas se pueden separar en selecciones encadenadas:

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

- Disyunciones se pueden explotar como uniones:

$$\sigma_{\theta_1 \vee \theta_2}(E) \equiv \sigma_{\theta_1}(E) \cup \sigma_{\theta_2}(E)$$

- Selecciones encadenadas son conmutativas (el orden no importa):

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Operaciones de conjunto

- Unión, intersección y junta son *conmutativas*:

$$E_1 \cup E_2 \equiv E_2 \cup E_1$$

- Y *asociativas*:

$$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$

Junta

La junta theta es *asociativa*:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

Esto permite, por ejemplo, cambiar el orden en que se hacen las juntas para evaluar primero las más restrictivas.

Heurísticas

Reglas prácticas que *reducen el espacio de búsqueda* sin calcular costos:

- **Realizar las selecciones lo antes posible:** *filtrar registros temprano* reduce la cantidad de datos que fluyen por el resto del plan.
- **Realizar las proyecciones temprano:** *descartar columnas innecesarias* reduce el tamaño de los registros intermedios.
- **Realizar las juntas más restrictivas primero:** *empezar por las juntas que producen menos registros* reduce el tamaño de las relaciones intermedias.

Estas heurísticas *no garantizan el plan óptimo*, pero *eliminan muchas opciones claramente malas*. Los optimizadores reales combinan decisiones basadas en heurísticas con decisiones basadas en costo para diferentes partes de la misma expresión.

Evaluación de expresiones de álgebra

Las expresiones de álgebra relacional se evalúan construyendo un *árbol de operadores*:

- Las **hojas** son las relaciones base (tablas).
- Los **nodos internos** son los operadores (σ , π , $\Join$, etc.).
- La **raíz** es el operador final cuyo resultado es la respuesta.

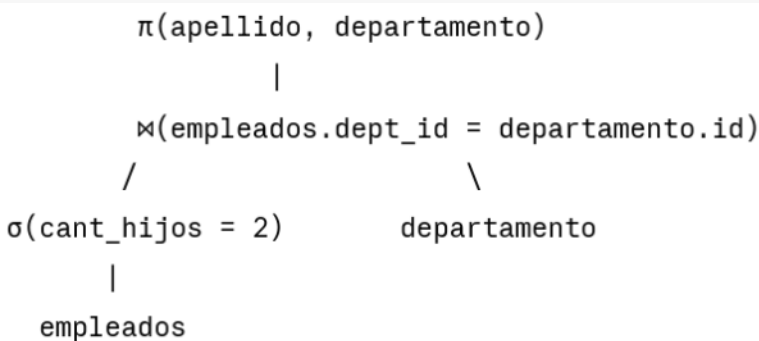
El árbol se evalúa *de abajo hacia arriba* (hojas $\rightarrow$ raíz). Cada nodo produce una relación intermedia que alimenta al nodo superior.

Para la consulta:

SQL

```
SELECT apellido, departamento
FROM empleados
INNER JOIN departamento ON empleados.dept_id = departamento.id
WHERE empleados.cant_hijos = 2
```

El árbol sería:



Materialización

Cuando los resultados intermedios *no caben en memoria*, se deben **materializar** (escribir a disco).

Esto implica costos adicionales:

- *Escribir* los bloques del resultado intermedio: $\lceil n_r / f_r \rceil$ bloques (donde n_r son los registros del resultado y f_r el factor de bloque).
- *Leer* esos bloques en el operador siguiente.

Pipelining

Es una *alternativa a la materialización*. Consiste en *encadenar operadores* para que los registros al nodo superior del árbol a medida que se van produciendo, sin esperar a que el

operador termine de escribir a disco.

Tiene como ventajas:

- Reducir la cantidad de archivos temporales (consumen poder de cómputo).
- Reducir latencia: se presentan los primeros resultados antes de terminar de procesar todo.

El pipelining *funciona cuando el operador superior puede empezar a trabajar con resultados parciales*.

Una selección seguida de una proyección a medida que σ emite registros, π los procesa de inmediato.

No funciona cuando el operador necesita ver *todos* los datos antes de producir resultados.

Un sort necesita todos los registros para poder ordenarlos.

Para $\pi_{\text{nombre}}(\sigma_{\text{edad}=27}(\text{Jugador}))$:

Con materialización:

- 1) File scan de **Jugador**, aplicar $\sigma_{\text{edad}=27} \rightarrow$ escribir resultado a disco.
- 2) Leer resultado de disco, aplicar $\pi_{\text{nombre}} \rightarrow$ escribir resultado final.

Con pipelining (entre σ y π):

- 1) File scan de **Jugador**, aplicar $\sigma_{\text{edad}=27} \rightarrow$ cada tupla que pasa el filtro se envía **directamente** a π_{nombre} .
- 2) Se ahorra toda la escritura/lectura del resultado intermedio.

Estimación de cardinalidades

Para calcular el costo de operadores que trabajan sobre relaciones intermedias (no tablas base), el optimizador necesita *estimar cuántas tuplas* produce cada paso. Estas estimaciones usan las estadísticas del catálogo y asumen **distribución uniforme** de valores.

Selección

Igualdad $\sigma_{A=a}(r)$:

$$\frac{n_r}{V(A, r)}$$

Si hay $V(A, r)$ valores distintos y distribución uniforme, cada valor tiene $n_r/V(A, r)$ registros.

Rango $\sigma_{A \leq v}(r)$:

- Si $v < \min(A, r)$: 0
- Si $v > \max(A, r)$: n_r
- Si no: $n_r \times \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$

Proporción del rango total que cubre la condición.

Conjunción $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$, donde s_i es la cardinalidad estimada de $\sigma_{\theta_i}(r)$:

$$n_r \times \frac{s_1 \times s_2 \times \dots \times s_n}{n_r^n}$$

Se asume independencia entre condiciones. Cada una filtra una fracción s_i/n_r .

Disyunción $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$:

$$n_r \times \left(1 - \left(1 - \frac{s_1}{n_r} \right) \times \left(1 - \frac{s_2}{n_r} \right) \times \dots \times \left(1 - \frac{s_n}{n_r} \right) \right)$$

Probabilidad de cumplir al menos una condición = 1 - probabilidad de no cumplir ninguna.

Junta

Producto cartesiano $r \times s$:

$$n_r \times n_s$$

Si θ implica clave en s : como máximo n_r (cada registro de r matchea con a lo sumo 1 de s).

Si θ implica clave en r : como máximo n_s .

Si θ no implica clave en ninguna:

$$\min \left(\frac{n_r \times n_s}{V(A, s)}, \frac{n_s \times n_r}{V(A, r)} \right)$$

Por cada registro de r se producen $n_s/V(A, s)$ matches en promedio. Se toma el mínimo entre las dos perspectivas.

Otras operaciones

- **Proyección (con distinct)** $\pi_A(r)$: $V(A, r)$
- **Agregación** $\gamma_A(r)$: $V(G, r)$ (una fila por cada valor del grupo)
- **Unión** $r \cup s$: $n_r + n_s$

- **Intersección** $r \cap s$: $\min(n_r, n_s)$
- **Left outer join** $r \Join_l s$: estimación de $r \Join s + n_r$
- **Full outer join** $r \Join_f s$: estimación de $r \Join s + n_r + n_s$